

TEIL 1: STRUKTOGRAMME (DIN 66261)

Bearbeitung der Aufgaben 3, 5, 6 und 12 aus dem Arbeitsblatt „Struktogramme“ (S. 12ff.).

Aufgabe 3 (S. 12): Niederschlagsmessungen auswerten (365 Tage)

Aufgabe: Ein eindimensionales Array `niederschlaege` (365 Tage, Index 0 bis 364) enthält Niederschläge in mm. Ermittle Höchstwert, Jahressumme und Tagesdurchschnitt und gib alle drei Werte aus.

summe = 0	
max_wert = niederschlaege[0]	
FÜR i VON 0 BIS 364 (Schrittweite 1):	
summe = summe + niederschlaege[i]	
Bedingung: niederschlaege[i] > max_wert	
[ja] max_wert = niederschlaege[i]	[nein] (keine Aktion)
durchschnitt = summe / 365	
Ausgabe: max_wert , summe , durchschnitt	

Aufgabe 5 (S. 13): Rabattsystem Paketdienst

Variablen: U (Umsatz lfd. Jahr), A (Anzahl bisherige Sendungen), P (Sendungspreis), B (Beförderungskosten), GP (Gesamtpreis).

Regeln: Ab 150 € Umsatz → 10 % Rabatt auf P ($r1 = 0.10 * P$). Ab 12. Sendung ($A \geq 11$) → halber Beförderungspreis ($r2 = 0.50 * B$). Die 24. Sendung ($A == 23$) → kostenlos. Es gilt jeweils die für den Kunden günstigste Regelung.

Ist die aktuelle Sendung die 24. im lfd. Jahr? (A == 23)	
[ja] (kostenlos) GP = 0	[nein] (Rabattvergleich) r1 = 0; r2 = 0
	U ≥ 150 ?
	[ja] r1 = 0.10 * P [nein] r1 = 0
	A ≥ 11 ?
	[ja] r2 = 0.50 * B [nein] r2 = 0
	r1 > r2 ?
	[ja] rabatt = r1 [nein] rabatt = r2
	GP = (P + B) - rabatt
Ausgabe: GP	

Aufgabe 6 (S. 13): Überprüfung und Korrektur des Mahnprogramms

1. Fehleranalyse des im Arbeitsblatt abgebildeten Struktogramms:

Das gegebene Struktogramm ist grob **fehlerhaft**:

- **Fehler 1 (Falsch-Mahnung bei zukünftigen Monaten):** Wenn $FM < AM$ nicht zutrifft (z.B. Rechnung fällig im Dezember $FM=12$, aktueller Monat September $AM=9$ desselben Jahres $FJ == AJ$), geht das Diagramm in den **nein**-Zweig und prüft $FJ \leq AJ$. Da $2026 \leq 2026$ wahr ist, wird **sofort eine Mahnung gedruckt**, obwohl die Rechnung erst in 3 Monaten fällig ist!
- **Fehler 2 (Unzulässige Jahreslogik):** Der genaue Fälligkeitstag zählt laut Vorgabe nicht; gemahnt wird erst im Folgemonat der Fälligkeit. Bei Fälligkeit im Vorjahr ($FJ < AJ$) ist die Rechnung stets fällig, unabhängig vom Monat.

Korrekte Mahn-Bedingung: Nicht bezahlt **UND** [$FJ < AJ$ **ODER** ($FJ == AJ$ **UND** $FM < AM$)].

2. Korrigiertes, verbessertes Struktogramm:

Ist die Rechnung bereits bezahlt? (bezahlt == wahr)		
[ja] (Keine Aktion - Vorgang erledigt)	[nein] - Fälligkeit prüfen:	
	Bedingung: $FJ < AJ$ ODER ($FJ == AJ$ UND $FM < AM$)	
	<table> <tr> <td>[ja] Mahnung auslösen!</td><td>[nein] (Frist noch nicht abgelaufen)</td></tr> </table>	[ja] Mahnung auslösen!
[ja] Mahnung auslösen!	[nein] (Frist noch nicht abgelaufen)	

Aufgabe 12 (S. 15): CARTRONIC GmbH – Höchster Tankstellenumsatz

Gegeben: 2 parallele Arrays der Länge 5: $TNAMEN = ["AGIP", "BP", "SHELL", "ESSO", "TOTAL"]$
 $TUMS = [11200.10, 23433.20, 7134.90, 14655.00, 4175.80]$

Aufgabe: Ermittle die Tankstelle mit dem höchsten Umsatz und gib deren Namen sowie den Umsatz aus.

max_pos = 0	
max_umsatz = TUMS[0]	
FÜR i VON 1 BIS 4 (Schrittweite 1):	
Bedingung: $TUMS[i] > max_umsatz$	
[ja] $max_umsatz = TUMS[i]$ $max_pos = i$	[nein] (keine Änderung)
Ausgabe: $TNAMEN[max_pos]$, max_umsatz (Ausgabe für die gegebenen Testdaten: "BP" mit 23.433,20 €)	

1. INSERTIONSORT (SORTIEREN DURCH EINFÜGEN)

Alltagsanalogie: Sortieren von Spielkarten auf der Hand. Man nimmt nacheinander jede unverschobene Karte und fügt sie an der passenden Stelle in die bereits sortierte linke Teilhand ein.

Ablauf am Beispiel [7, 3, 5, 2] :

- *Index 1 (Wert 3):* Vergleich mit 7 → 7 rückt nach rechts → 3 an Pos. 0 → [3, 7, 5, 2]
- *Index 2 (Wert 5):* Vergleich mit 7 → 7 rückt nach rechts; $5 \geq 3$ → Stopp → [3, 5, 7, 2]
- *Index 3 (Wert 2):* Wandert ganz nach vorne → [2, 3, 5, 7] (fertig sortiert)

```
def insertionsort(liste):
    for i in range(1, len(liste)):
        wert = liste[i]
        j = i - 1
        while j >= 0 and liste[j] > wert:
            liste[j + 1] = liste[j]
            j -= 1
        liste[j + 1] = wert
    return liste
```

Verständnisaufgabe für den Kurs: Gegeben ist [9, 4, 6, 1] . Zustand nach jedem Durchlauf der äußeren Schleife?
Musterlösung: $i=1$: [4, 9, 6, 1] | $i=2$: [4, 6, 9, 1] | $i=3$: [1, 4, 6, 9] .

2. QUICKSORT (SORTIEREN DURCH ZERLEGEN – DIVIDE & CONQUER)

Prinzip: Ein Referenzelement (Pivot) wird gewählt. Die Restliste wird in zwei Partitionen aufgeteilt (alle kleineren links, alle größeren rechts). Rekursiver Aufruf für beide Hälften.

Ablauf am Beispiel [5, 2, 8, 3, 9, 1] (**Pivot = Mitte = 3**):

- Partitionierung: Links (< 3): [2, 1] | Mitte ($= 3$): [3] | Rechts (> 3): [5, 8, 9]
- Rekursion Links [2, 1] (Pivot 1): [] + [1] + [2] = [1, 2]
- Rekursion Rechts [5, 8, 9] : analog → [5, 8, 9]
- Zusammenfügen: [1, 2] + [3] + [5, 8, 9] = [1, 2, 3, 5, 8, 9]

```
def quicksort(liste):
    if len(liste) <= 1:
        return liste
    pivot = liste[len(liste) // 2]
    links = [x for x in liste if x < pivot]
    mitte = [x for x in liste if x == pivot]
    rechts = [x for x in liste if x > pivot]
    return quicksort(links) + mitte + quicksort(rechts)
```

Verständnisaufgabe für den Kurs: Warum ist die Wahl des Pivot-Elements bei Quicksort so entscheidend?
Musterlösung: Halbiert das Pivot die Liste stets, ist der Rekursionsbaum optimal flach (Tiefe $\log_2 n \rightarrow O(n \log n)$). Wählt man unglücklich immer das Minimum oder Maximum, zerfällt die Teilliste in 0 und $n-1$ Elemente → Baumtiefe n führt zum Worst Case $O(n^2)$.

1. LAUFZEITMESSUNG MIT PYTHON & PROBLEMATIK

Messung mit Python: Nutzung von `time.process_time()` (misst reine CPU-Zeit des Prozesses, schließt Leerlauf aus):

```
from time import process_time
t1 = process_time()
L_sortiert = sortierverfahren(L)
t2 = process_time()
rechenzeit = t2 - t1
```

Problematik realer Laufzeitmessungen: Gemessene Sekunden sind kein universelles Maß für Algorithmen. Sie hängen von Rechnerhardware, CPU-Taktung (Turbo-Boost / Thermal Throttling), Cache-Zuständen und konkurrierenden Hintergrundprozessen des Betriebssystems ab.

2. BEARBEITUNG DER AUFGABE 2 (INF-SCHULE)

(a) Warum schwanken die Messergebnisse leicht bei wiederholten Läufen derselben Liste?

Antwort: Ursache sind CPU-Caching (Cold Cache vs. Warm Cache), Cache-Misses, minimale Betriebssystem-Interrupts und die automatische Speicherverwaltung (Garbage Collection) von Python.

(b) Warum können eigene Messwerte in einem völlig anderen Zahlenbereich liegen?

Antwort: Hardwareabhängigkeit! Ein moderner Desktop-Prozessor mit hoher Single-Thread-Leistung führt dieselben Operationen in wenigen Millisekunden aus, während ein älterer Schul-PC oder Laptop im Energiesparmodus deutlich länger braucht.

(c) Erkennung des weiteren Hintergrundprozesses & resultierende Problematik:

Antwort: Bei Durchlauf 4 steigt die Zeit plötzlich sprunghaft von ca. 3,41 s auf 5,37 s an. Dies zeigt, dass Multitasking und fremde Prozesse Rechenkapazitäten verdrängen. Physikalische Laufzeiten sind daher nicht objektiv reproduzierbar.

3. THEORETISCHE AUFWANDS- & KOSTENANALYSE

- **Problemgröße n :** Anzahl der Datensätze / Elemente in der Liste.
- **Modellierung der Kosten $T(n)$:** Als Kennmaß dient die Anzahl der *elementaren Datensatzvergleiche*.
- **Fallunterscheidung:** Best Case (minimaler Aufwand), Worst Case (maximale Obergrenze), Average Case (Durchschnitt).

Algorithmus	Best Case	Worst Case	Average Case	Begründung & mathematische Einordnung
Insertionsort	$T(n) = n - 1$ $O(n)$	$T(n) = \frac{1}{2} n(n-1)$ $O(n^2)$	$T(n) \approx \frac{1}{4} n^2$ $O(n^2)$	Im <i>Best Case</i> (bereits aufsteigend sortiert) schlägt der innere Vergleich sofort fehl ($n-1$ Vergleiche). Im <i>Worst Case</i> (umgekehrt sortiert) muss jedes Element i bis an Position 0 wandern.
Quicksort	$T(n) \approx n \cdot \log_2(n)$ $O(n \log n)$	$T(n) = \frac{1}{2} n(n-1)$ $O(n^2)$	$T(n) \approx 1,39 n \log_2 n$ $O(n \log n)$	Im <i>Best/Average Case</i> teilt das Pivot die Liste stets symmetrisch (Baumtiefe $\log_2 n \times n$ Vergleiche). Im <i>Worst Case</i> (Pivot ist immer Minimum/Maximum) zerfällt das Problem in n Stufen wie bei naiven Verfahren.